PRICEPILOT AI

Dynamic Pricing Optimization & Revenue Intelligence System

MILESTONE 4 DOCUMENTATION

Testing, Deployment & Final Demonstration | Weeks 7–8

Submitted By
Aditya Raj Pandey

1. Project Overview

Milestone 4: Weeks 7–8 — Testing, Deployment & Documentation

PricePilot AI is an AI-assisted dynamic pricing and revenue intelligence platform designed to support business pricing decisions using product information, costs, inventory, competitor prices, promotions, demand patterns and historical sales.

Milestone 4 consolidates work from the earlier milestones into a testable, deployable and demonstrable application. It focuses on model validation, recommendation quality, API and UI reliability, performance, containerization, deployment readiness, final documentation and an end-to-end demonstration.

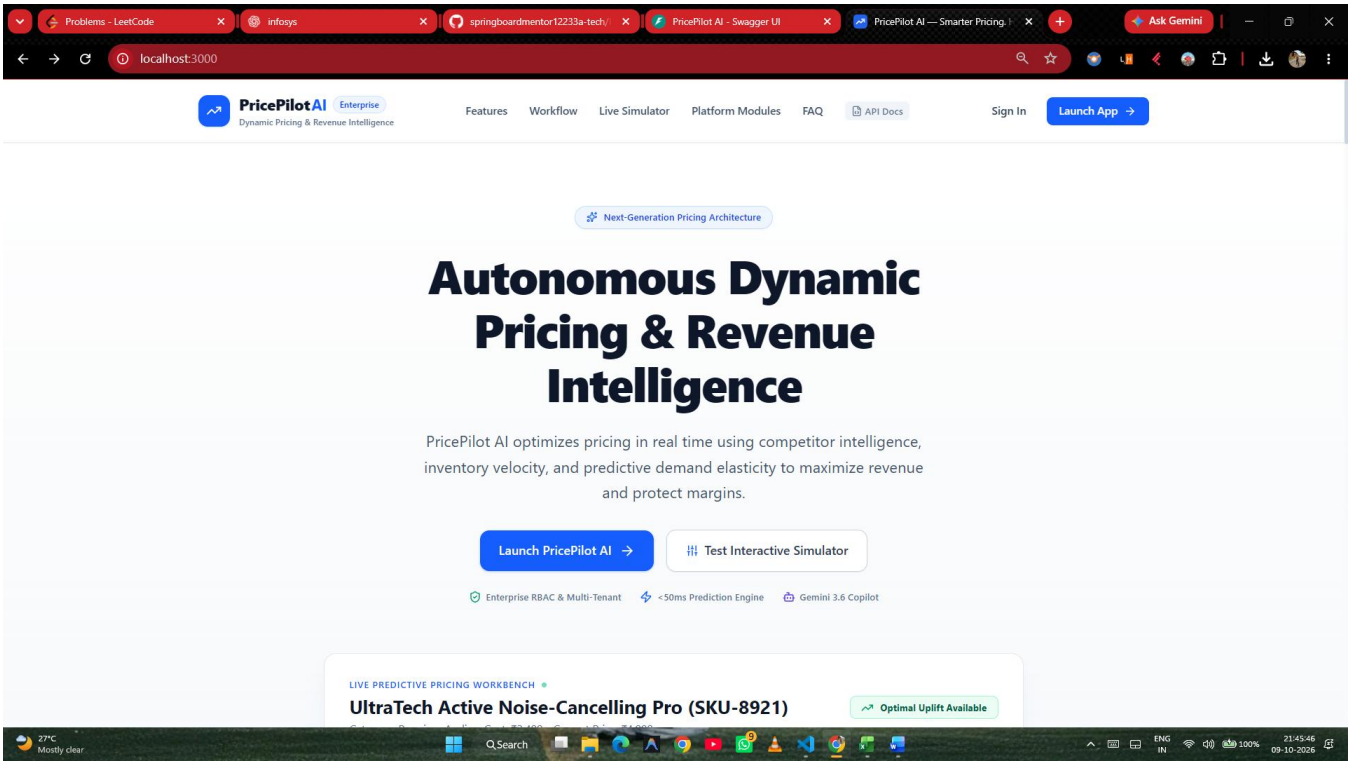
Objectives

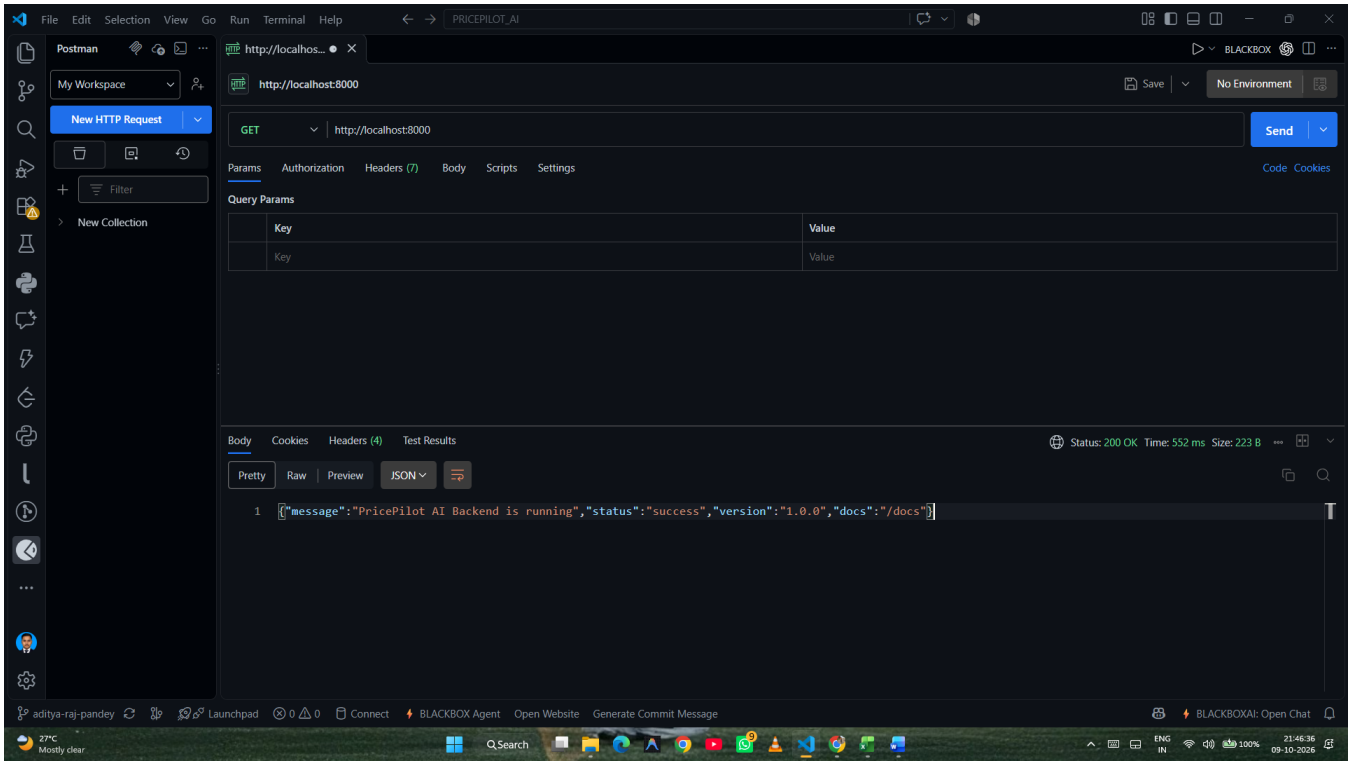
- Validate prediction quality using appropriate holdout data and documented metrics.
- Test pricing recommendations and business calculations for consistency.
- Verify React frontend, FastAPI backend, PostgreSQL and ML inference as one workflow.
- Improve dashboard responsiveness, loading states, error handling and usability.
- Package services with Docker and prepare cloud deployment configuration.
- Prepare final technical documentation, user guidance and a repeatable demonstration.

Expected outputs

Output	Evidence expected
ML inference	Artifact loads; fitted model returns finite prediction.
Backend	Health check, route tests, validation and errors recorded.
Frontend	Production build and critical user journeys pass.
Deployment	Container builds, services start and smoke tests pass.
Handover	Setup guide, architecture, test results and limitations documented.

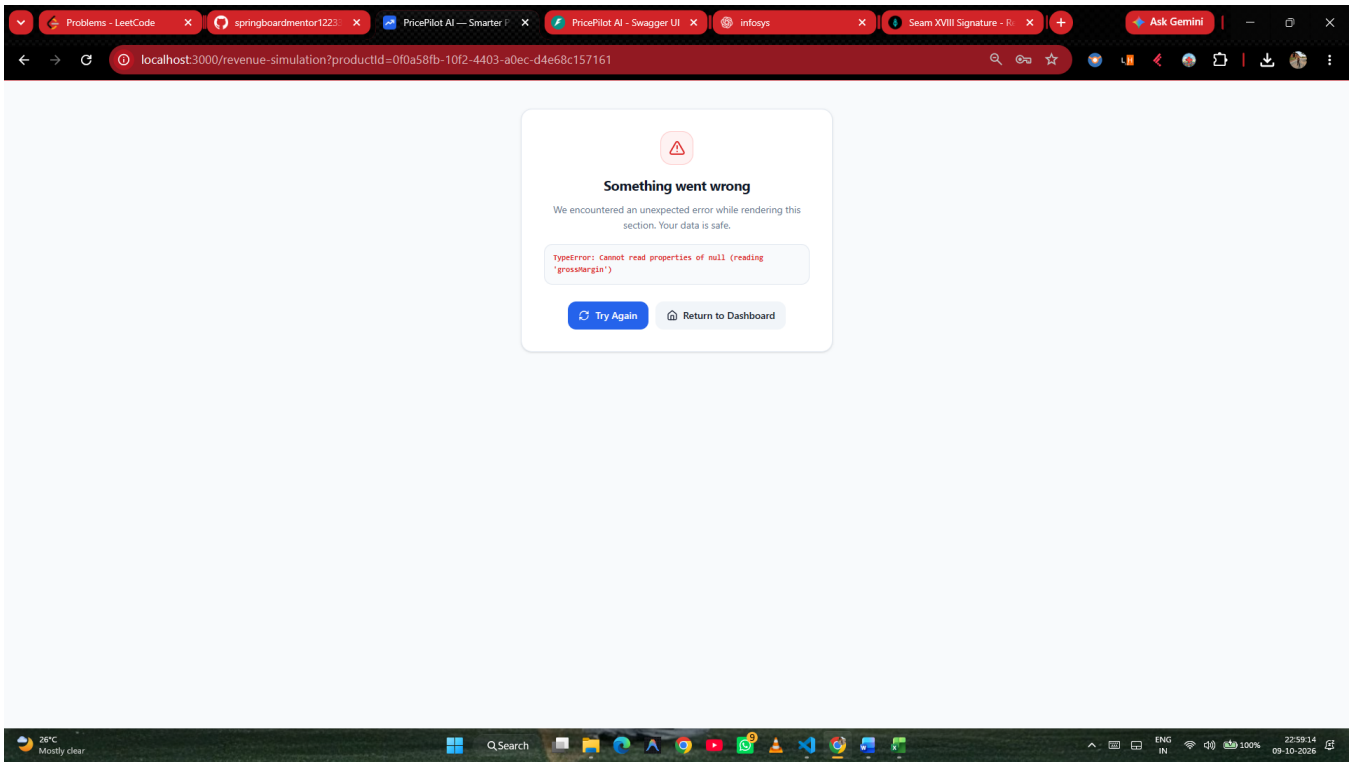
Evidence policy
This document is a test and release framework. Mark work complete only after collecting execution evidence.

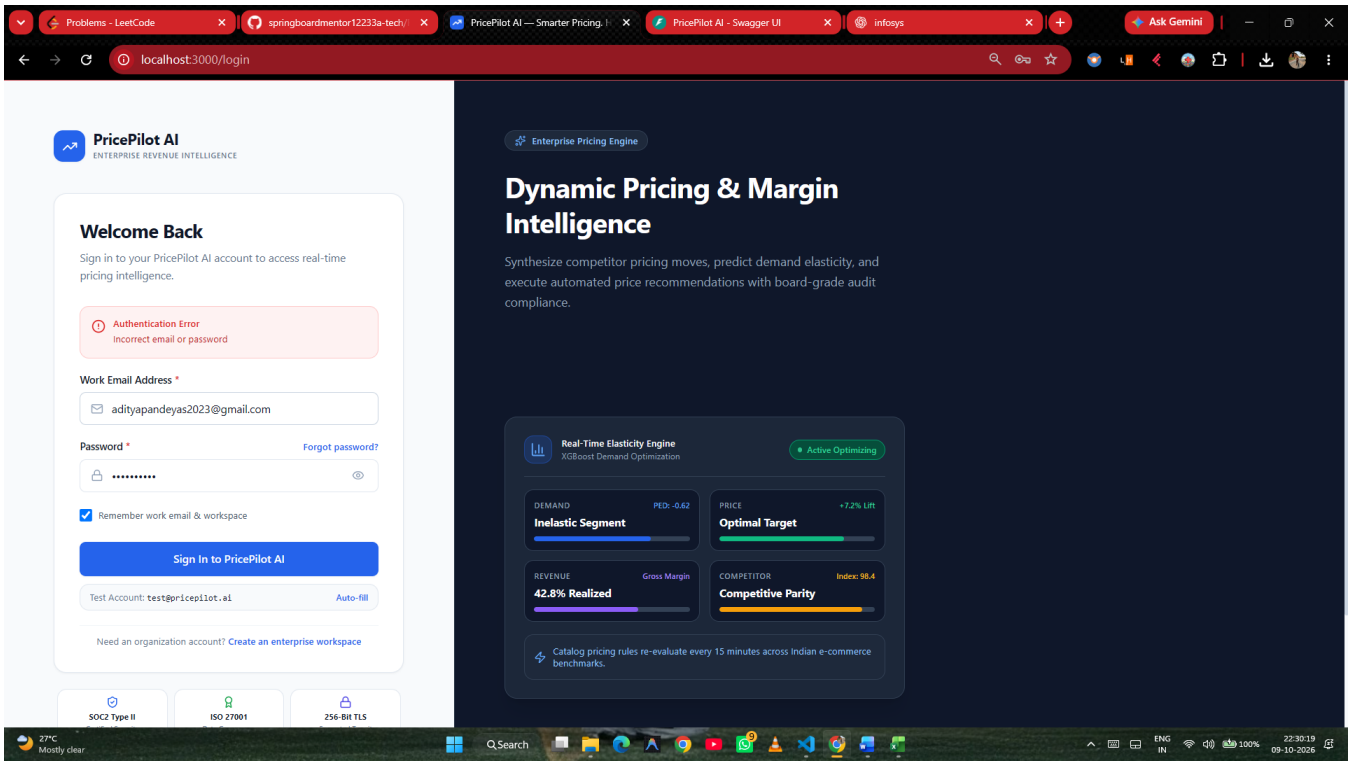


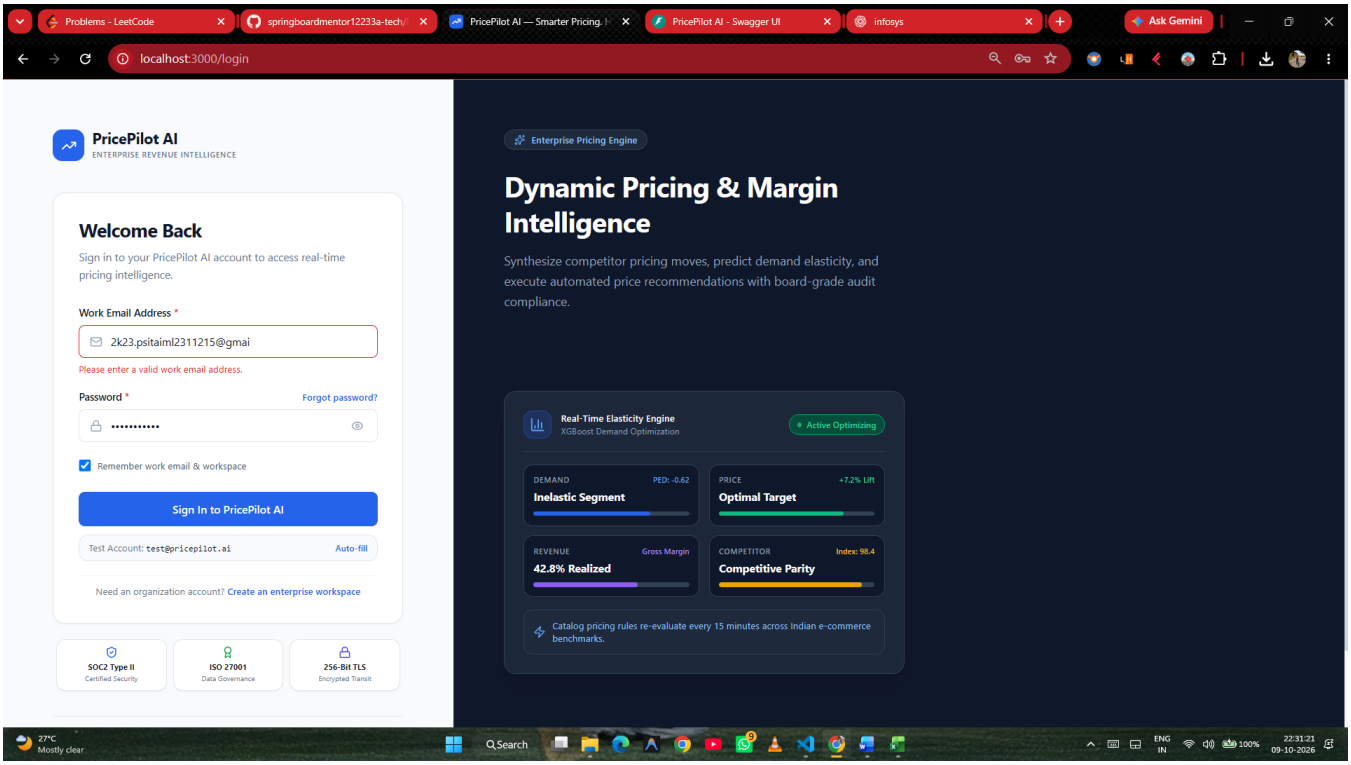


Test Cases(Authentication)

- Rate Limiter Restricts the Repeated Invalid login attempt




Problems - LeetCode
springboardmentor12233a-tech
PricePilot AI — Smarter Pricing
PricePilot AI - Swagger UI
infosys
Ask Gemini
localhost:3000/login
PricePilot AI
ENTERPRISE REVENUE INTELLIGENCE
Welcome Back
Sign in to your PricePilot AI account to access real-time pricing intelligence.
Authentication Error
Incorrect email or password
Work Email Address *
adityapandeyas2023@gmail.com
Password *
Forgot password?
Remember work email & workspace
Sign In to PricePilot AI
Test Account: test@pricepilot.ai
Auto-fill
Need an organization account? Create an enterprise workspace
SOC2 Type II
ISO 27001
256-Bit TLS
Enterprise Pricing Engine
Dynamic Pricing & Margin Intelligence
Synthesize competitor pricing moves, predict demand elasticity, and execute automated price recommendations with board-grade audit compliance.
Real-Time Elasticity Engine
XGBoost Demand Optimization
Active Optimizing
DEMAND
PED: -0.62
Inelastic Segment
PRICE
+7.2% Lift
Optimal Target
REVENUE
Gross Margin
42.8% Realized
COMPETITOR
Index: 98.4
Competitive Parity
Catalog pricing rules re-evaluate every 15 minutes across Indian e-commerce benchmarks.
27°C
Mostly clear
Search
ENG
IN
100%
22:30:19
09-10-2026


Problems - LeetCode
springboardmentor12233a-tech
PricePilot AI — Smarter Pricing
PricePilot AI - Swagger UI
infosys
Ask Gemini
localhost:3000/login
PricePilot AI
ENTERPRISE REVENUE INTELLIGENCE
Welcome Back
Sign in to your PricePilot AI account to access real-time pricing intelligence.
Work Email Address *
2k23.psitaiml2311215@gmai
Please enter a valid work email address.
Password *
Forgot password?
Remember work email & workspace
Sign In to PricePilot AI
Test Account: test@pricepilot.ai
Auto-fill
Need an organization account? Create an enterprise workspace
SOC2 Type II
Certified Security
ISO 27001
Data Governance
256-Bit TLS
Encrypted Transit
Enterprise Pricing Engine
Dynamic Pricing & Margin Intelligence
Synthesize competitor pricing moves, predict demand elasticity, and execute automated price recommendations with board-grade audit compliance.
Real-Time Elasticity Engine
XGBoost Demand Optimization
Active Optimizing
DEMAND
PED: -0.62
Inelastic Segment
PRICE
+7.2% Lift
Optimal Target
REVENUE
Gross Margin
42.8% Realized
COMPETITOR
Index: 98.4
Competitive Parity
Catalog pricing rules re-evaluate every 15 minutes across Indian e-commerce benchmarks.
27°C
Mostly clear
Search
ENG
IN
100%
22:31:21
09-10-2026

2. System Architecture & Integration

The application separates the presentation layer, HTTP API, business services, model inference and persistent data. The browser communicates through backend APIs and must never connect directly to PostgreSQL or contain secret provider keys.

Layer	Responsibility	Verification
React + Vite	Routes, forms, dashboards, charts, tables and assistant UI.	Rendering, validation, accessibility, responsive layout.
API client/proxy	HTTP requests, auth headers and response handling.	Correct target, status handling, no secrets exposed.
FastAPI	Routes, schemas, authorization and exception handling.	OpenAPI contract, permissions, status codes.
Business services	Pricing, competitor analysis, revenue and AI orchestration.	Business rules, calculation correctness.
ML inference	Feature construction, saved preprocessing and XGBoost.	Feature schema, artifact compatibility, finite output.
PostgreSQL	Organizations, users, products, inventory, competitor and sales data.	Migrations, constraints, transactions, tenant isolation.

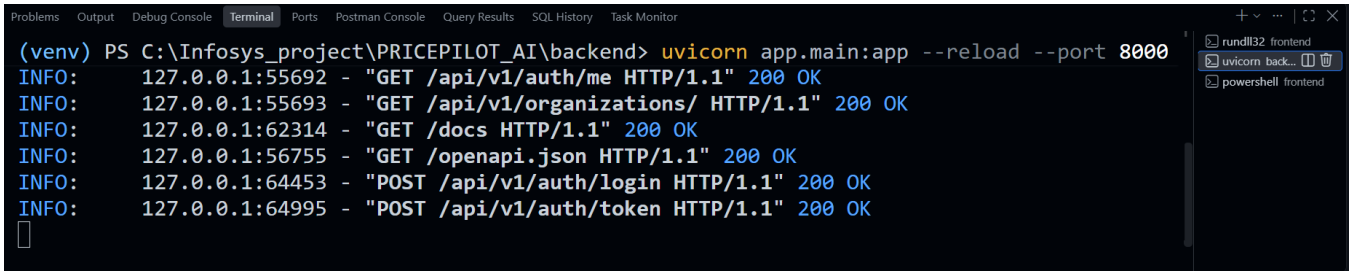
Request flow

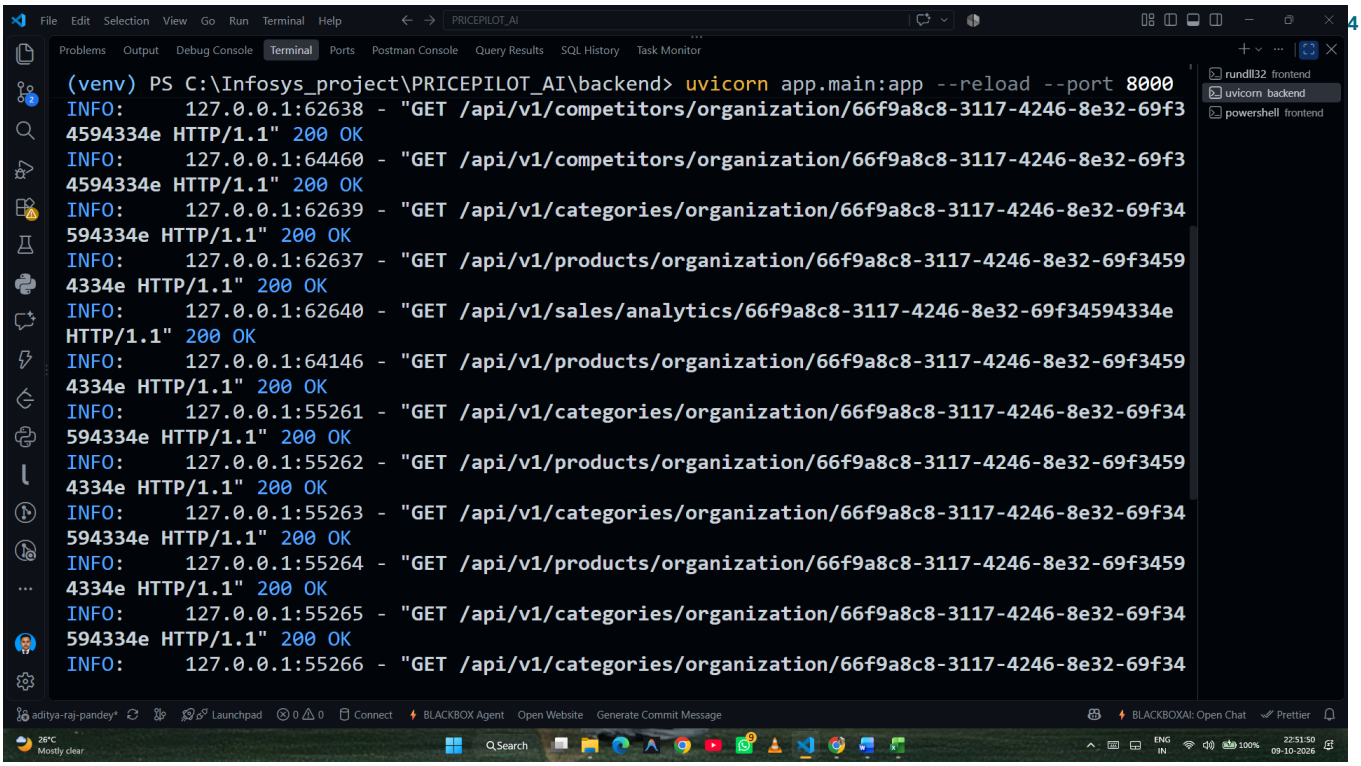
1. User signs in and enters an authenticated workflow.
2. Frontend sends a validated request through the API client.
3. FastAPI validates input and verifies access permissions.
4. Services retrieve organization/product context from PostgreSQL.
5. Feature generation creates the expected 26 model features and applies the saved preprocessor.
6. Fitted XGBoost returns a demand estimate; business logic interprets it.
7. API returns structured JSON and frontend renders success, empty, loading or error states.

Critical A serialized dictionary containing an unfitted XGB Regressor is not a functioning inference artifact.

Configuration principles

- Separate local and production URLs.
- Keep secrets in environment variables or a secret manager.
- Use the live OpenAPI schema as the API contract.
- Exclude .env, credentials, virtual environments and local databases from Git.





3. Machine Learning Model Validation

The selected demand model is a tuned XGBoost regressor. Validation covers artifact integrity, preprocessing compatibility, input schema, prediction execution, numerical output and holdout performance.

Previously recorded benchmark

The recorded final tuned XGBoost experiment reported test MAE 12.4338, RMSE 17.2366 and R^2 0.8527. These values are an experiment reference, not a guarantee that the deployed API will reproduce them. Reproduction requires the same model, preprocessing, data split and evaluation procedure.

Metric	Recorded test result	Meaning
MAE	12.4338	Mean absolute error in demand units.
RMSE	17.2366	Error metric that penalizes larger misses more strongly.
R^2	0.8527	Explained-variance measure; not an accuracy percentage.

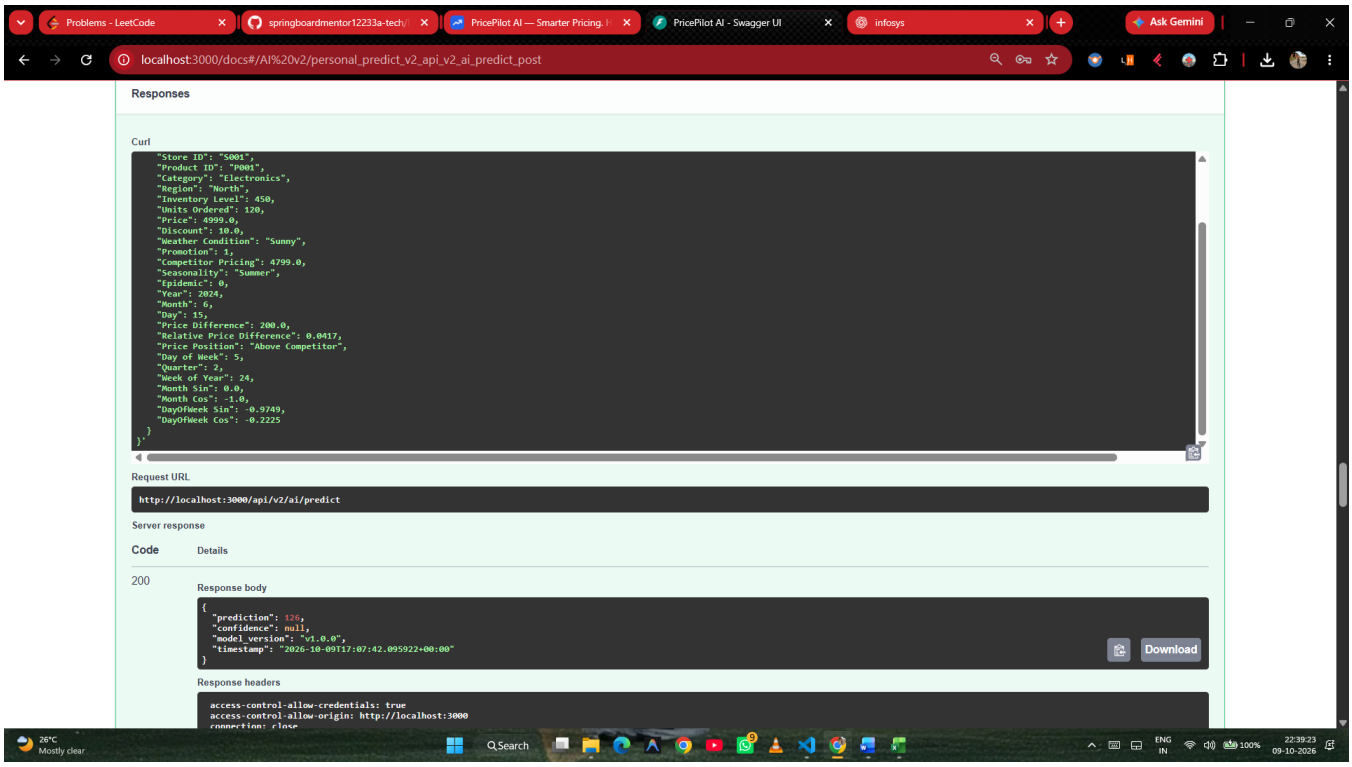
Required tests

- Load artifact and confirm keys model, preprocessor, features, model_name and version.
- Confirm fitted state by accessing the booster or making a real prediction.
- Validate the 26 feature names and their order.
- Transform a one-row DataFrame and verify processed shape matches model expectations.
- Reject missing fields, malformed categories and invalid numeric values.
- Ensure predictions are finite and failures return controlled errors.
- Compare API prediction against direct artifact inference for equivalent features.

Leakage and limits

Units Sold and Revenue are outcomes and should not be included as same-row demand input features. Inventory Level and Units Ordered should only be used if available at prediction time. This model is not inherently a recursive multi-day time-series forecaster.

Known failure mode A previous direct test raised NotFittedError even though the artifact dictionary loaded. Successful deserialization alone does not prove inference works.



4. ML API & Prediction Contract Testing

Test the API endpoint separately from a standalone model script. The previously verified prediction route is POST /api/v2/ai/predict; confirm the live Swagger/OpenAPI schema before relying on any additional endpoint.

Illustrative business-level payload

```
{
 "store_id": "S001",
 "product_id": "P001",
 "category": "Electronics",
 "region": "North",
 "inventory_level": 120,
 "units_ordered": 85,
 "price": 4999,
 "discount": 10,
 "weather_condition": "Clear",
 "promotion": 1,
 "competitor_pricing": 4799,
 "seasonality": "Winter",
 "epidemic": 0,
 "date": "2026-10-09"
}
```

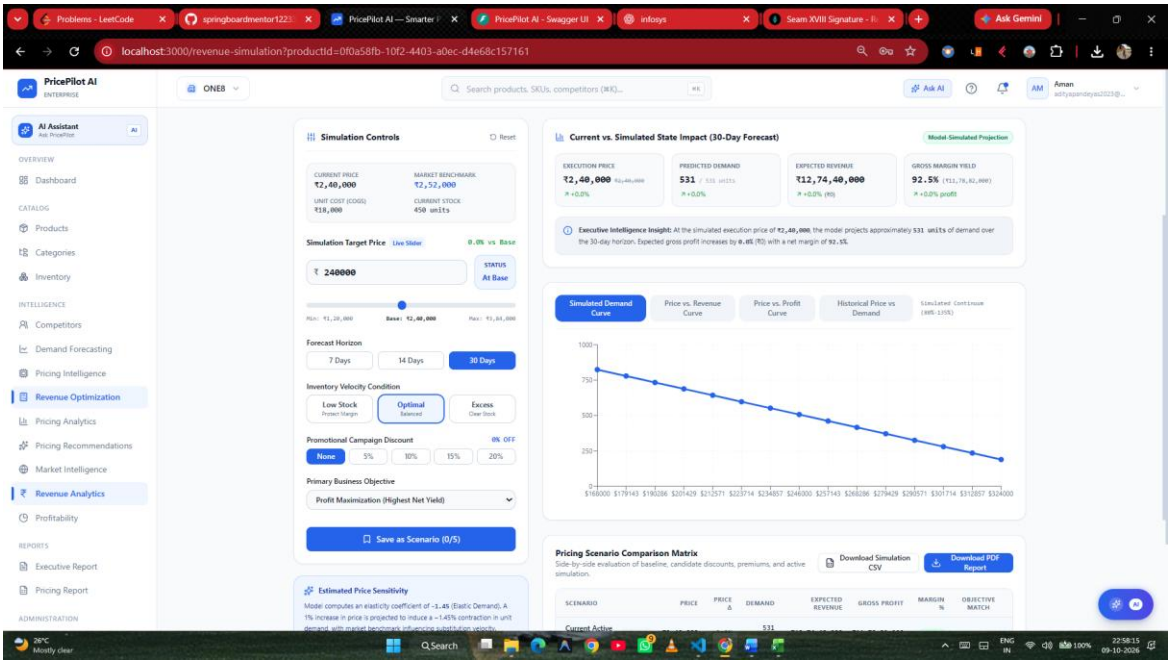
This sample is illustrative, not a guaranteed endpoint schema. The live API may require engineered features explicitly or different field names. Use the exact request schema shown in /docs.

Test	Expected behavior
Valid request	Success response with documented prediction field.
Missing required field	422 or documented validation error.
Invalid numeric value	Controlled validation error; no traceback to client.
Unknown category	Handled by schema/encoder policy without crash.
Artifact missing	Safe 5xx response and useful server-side log.
NaN/Infinity prediction	Reject invalid result; log the failure.
Unauthorized call	401/403 according to route policy.
Repeated same input	Stable output if inference is deterministic.

Execution steps

8. Start backend using backend/venv.
9. Open /docs and expand POST /api/v2/ai/predict.
10. Submit a valid request using the live schema.
11. Record status, JSON response, latency and server log.
12. Repeat with invalid inputs.

13. Compare one result with direct inference on the same engineered row.



5. Backend, Database & Security Testing

Backend route tests

- Check GET / and GET /health.
- Verify registration, login, current-user lookup, organization membership and roles.
- Test product/category create, list, update and delete routes where implemented.
- Test inventory reads and updates with boundary values.
- Test competitor creation, matching, price ingestion and history.
- Test sales analytics, pricing and AI routes against live OpenAPI.
- Ensure database outages and constraint errors return controlled responses.

Database integrity

Area	Checks
Migrations	Fresh upgrade succeeds and schema matches ORM models.
Relationships	Foreign keys prevent orphan records.
Uniqueness	Email, organization slug and SKU constraints work.
Transactions	Multi-step changes commit or roll back consistently.
Tenant isolation	Organization A cannot read or mutate B's data.
Audit fields	Created/updated timestamps are populated consistently.
Deletion	Soft-delete/cascade behavior matches documented design.

Security checks

- Passwords are hashed; secrets and tokens are not logged.
- Backend enforces authorization on every protected resource.
- CORS origins are explicit and environment-appropriate.
- User inputs and identifiers are validated.
- Errors do not expose stack traces, SQL, secrets or filesystem paths.
- Secrets, local databases and environment folders are excluded from Git.

Security principle Frontend route guards are not a security boundary. Authorization must be enforced server-side.

6. Frontend Functional & Visual Testing

A successful production build confirms compilation but does not prove every screen or API workflow functions correctly.

Screen	Checks
Landing page	Navigation, sections, simulator CTA, docs link and responsive hero.
Login/register	Validation, pending state, errors, success redirect and logout.
Dashboard	Organization-scoped KPIs, charts and loading/empty/error states.
Products/categories	Create/view/edit/delete, validation and associations.
Inventory	Stock update, low-stock behavior and invalid quantities.
Competitors	Records, product mapping, price comparison and timestamps.
Pricing/simulation	Candidate price, demand/revenue/profit calculations and constraints.
Forecasting/analytics	Filters, chart axes, trends and actual-vs-predicted distinction.
Reports/BI	Period selection, organization scope, timestamps and exports.
AI assistant	Readable assistant output, retry, Markdown and no raw JSON display.

Visual and responsive review

- Test desktop, tablet and mobile viewports.
- Confirm Tailwind CSS is loaded and intended styles are applied.
- Resolve console errors, missing icon imports and undefined components.
- Check charts, tables, dialogs, dropdowns and sidebars for overflow.
- Verify keyboard navigation, focus states, labels and readable validation messages.
- Use consistent ₹ formatting for India-configured financial values.

Technical checks

Run npm run build. Test direct navigation and refresh on protected routes, expired sessions, logout, failed API calls and network interruption. Confirm Vite proxy and API base URL point to the intended backend.

7. End-to-End Workflow & Acceptance Testing

Workflow	Steps	Acceptance criteria
Authentication	Sign in → profile → dashboard → sign out.	Session and protected routes behave correctly.
Product	Create product → assign category → edit → reload.	Persisted data remains consistent.
Inventory	Open product → update stock → retrieve inventory.	Validated quantity persists and displays correctly.
Competitor	Add competitor → match product → record price → compare.	Correct product, price and source/date are shown.
Demand prediction	Submit valid features → API → render prediction.	Finite response and correct frontend mapping.
Price scenario	Choose candidate price → estimate demand → calculate profit.	Correct formula and visible assumptions.
BI report	Choose period → fetch organization data → export.	Scope, timestamp and totals match source data.
AI assistant	Ask question → receive provider output → render.	Readable answer; errors recover cleanly.

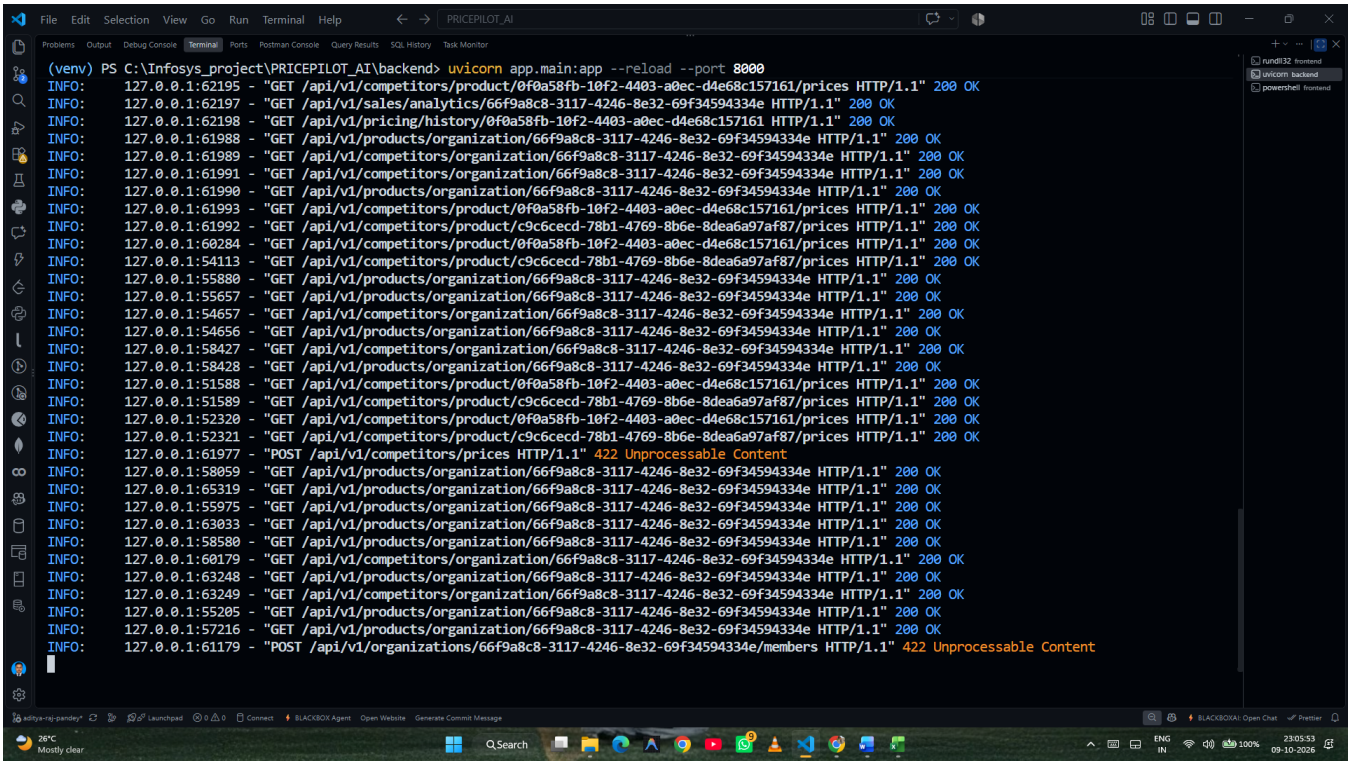
Calculation sanity checks

For a basic scenario, estimated revenue equals candidate selling price multiplied by predicted units. Gross profit must use the explicitly defined unit cost and price assumptions; disclose whether discounts, fees, returns, shipping or taxes are included. Gross margin is gross profit divided by revenue when revenue is positive.

Interpretation A model estimate does not prove a price change causes demand to change. Label simulated values and assumptions clearly.

Acceptance criteria

- No critical path fails in the demonstration scenario.
- Invalid input does not crash the application.
- Frontend values match API/database results for test records.
- Charts/reports identify period and data source.
- Historical, predicted and simulated values are distinct.
- Incomplete features and limitations are disclosed.



8. Performance, Reliability & Usability

Metric	Measurement	Reporting notes
Inference latency	Measure API prediction duration; report median/p95.	Include hardware and warm/cold state.
API latency	Measure key endpoints and concurrency.	Separate DB/provider/model where possible.
Dashboard load	Browser performance tools.	Record browser, viewport and cache.
Bundle size	Inspect Vite production build output.	Consider lazy loading large modules.
Error rate	Count failures during repeatable tests.	Group by endpoint and status.
Concurrency	Controlled load test in staging.	Monitor CPU, memory, connections and timeouts.

Optimization actions

- Use pagination and server-side filtering for large datasets.
- Debounce search and avoid redundant API requests.
- Lazy-load heavy routes or chart/report modules where beneficial.
- Index database columns based on measured query plans.
- Set external AI provider timeouts and show recoverable errors.
- Use structured logs and correlation IDs to connect UI and API failures.

Failure recovery tests

Simulate database unavailability, AI-provider rate limits, expired tokens, request timeouts and a missing model artifact. Users should receive clear recoverable messages; technical details belong in protected logs.

Performance claims Do not claim sub-50 ms prediction, high concurrency or uptime targets unless measured in the actual deployment environment.

FINAL REPORT



Test ID	Module	Method	Endpoint	Operation	Priority	Auth Required	Test Status	HTTP Status	Actual Result / Notes
API-001	Authentication	POST	/api/v1/auth/register	Register	High	Yes	Passed	200	200 OK (existing workbook)
API-002	Authentication	POST	/api/v1/auth/login	Login	High	Yes	Passed	-	-
API-003	Authentication	POST	/api/v1/auth/token	Token	High	No / verify route policy	Passed	200	200 OK (existing workbook)
API-004	Authentication	GET	/api/v1/auth/me	Current user	High	Yes	Passed	200	200 OK (existing workbook)
API-005	Users	GET	/api/v1/users/	List users	Medium	Yes	Passed	200	-
API-006	Users	GET	/api/v1/users/{user_id}	Get user by ID	Medium	Yes	Passed	200	-
API-007	Users	PUT	/api/v1/users/{user_id}	Update user	Medium	Yes	Passed	200	-
API-008	Organizations	GET	/api/v1/organizations/	List organizations	Medium	Yes	Passed	200	-
API-009	Organizations	POST	/api/v1/organizations/	Create organization	Medium	Yes	Passed	200	-
API-010	Organizations	GET	/api/v1/organizations/{org_id}	Get organization by ID	Medium	Yes	Passed	200	-
API-011	Organizations	POST	/api/v1/organizations/{org_id}/members	Add organization member	Medium	Yes	Passed	200	-
API-012	Categories	POST	/api/v1/categories/	Create category	Medium	Yes	Passed	200	-
API-013	Categories	GET	/api/v1/categories/organization/{org_id}	List organization categories	Medium	Yes	Passed	200	-
API-014	Products	POST	/api/v1/products/	Create product	Medium	Yes	Passed	200	-
API-015	Products	GET	/api/v1/products/organization/{org_id}	List organization products	Medium	Yes	Passed	200	-
API-016	Products	GET	/api/v1/products/{product_id}	Get product by ID	Medium	Yes	Passed	200	-
API-017	Products	PUT	/api/v1/products/{product_id}	Update product	Medium	Yes	Passed	200	-
API-018	Products	DELETE	/api/v1/products/{product_id}	Delete product	High	Yes	Passed	200	-
API-019	Products	POST	/api/v1/products/{product_id}/variants	Create product variant	Medium	Yes	Passed	200	-
API-020	Products	GET	/api/v1/products/{product_id}/inventory	Get inventory	Medium	Yes	Passed	200	-
API-021	Products	PUT	/api/v1/products/{product_id}/inventory	Update inventory	Medium	Yes	Passed	200	-
API-022	Competitors	POST	/api/v1/competitors/	Create competitor	Medium	Yes	Passed	200	-
API-023	Competitors	GET	/api/v1/competitors/organization/{org_id}	List organization competitors	Medium	Yes	Passed	200	-
API-024	Competitors	PUT	/api/v1/competitors/{competitor_id}	Update competitor	High	Yes	Passed	200	-
API-025	Competitors	POST	/api/v1/competitors/match	Match competitors	High	Yes	Failed	422	Request validation failed; inspect response for required fields
API-026	Competitors	POST	/api/v1/competitors/prices	Submit competitor prices	Medium	Yes	Not Run	200*	-
API-027	Competitors	GET	/api/v1/competitors/product/{product_id}/prices	Get competitor prices	Medium	Yes	Not Run	200*	-
API-028	Pricing Engine	POST	/api/v1/pricing/predict	Predict pricing	High	Yes	Not Run	200*	-
API-029	Pricing Engine	POST	/api/v1/pricing/recommendations	Generate recommendations	High	Yes	Not Run	200*	-
API-030	Pricing Engine	POST	/api/v1/pricing/recommendations/{recommendation_id}/apply	Apply recommendation	High	Yes	Not Run	200*	-
API-031	Pricing Engine	GET	/api/v1/pricing/history/{product_id}	Get pricing history	High	Yes	Not Run	200*	-
API-032	Sales	POST	/api/v1/sales/	Record sale	Medium	Yes	Not Run	200*	-
API-033	Sales	GET	/api/v1/sales/analytics/{organization_id}	Get sales analytics	Medium	Yes	Not Run	200*	-
API-034	AI v2	POST	/api/v2/ai/gemini	Gemini AI request	High	No / verify route policy	Passed	200	200 OK (user-provided Swagger result)
API-035	AI v2	POST	/api/v2/ai/predict	AI prediction	High	Yes	Passed	200	200 OK (user-provided Swagger result)
API-036	System	GET	/	Root endpoint	Medium	No	Passed	200	200 OK (existing workbook)
API-037	System	GET	/health	Health check	Medium	No	Passed	200	200 OK (existing workbook)

Submitted By

Aditya Raj Pandey

Thank You